

# Pattern Matching in Pizarnik

V. E. McHale

April 26, 2025

## Contents

|          |                                      |          |
|----------|--------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                  | <b>1</b> |
| <b>2</b> | <b>Extensible Pattern Matches</b>    | <b>1</b> |
| 2.1      | Reusing Pattern-Match Arms . . . . . | 2        |
| 2.2      | Typesafe Head . . . . .              | 3        |
| 2.3      | Pattern Terms . . . . .              | 3        |

## 1 Introduction

Pizarnik’s approach to pattern-matching is based on commitment to the idea of pattern-matching as inverse[1] and suggestive notation from linear logic (&, “with”, is dual to disjunction,  $\oplus$ ) [4].

## 2 Extensible Pattern Matches

Suppose we define an algebraic data type

```
type Bool = `true  $\oplus$  `false;
```

The literal ``true` pushes a tag<sup>1</sup> ``true` onto the stack. We assign it the type `-- `true`. Its inverse ``true-1` pops a value of type ``true` off the stack and hence the atom ``true-1` has type ``true --` (inverse exchanges left and right).

In linear logic, one has  $(G \oplus H)^\perp = G^\perp \& H^\perp$ . To pop off a value of type `Bool = { `true  $\oplus$  `false }`, one writes `{ `true-1 & `false-1 }` in Pizarnik. Or-patterns in other programming languages work like linear logic’s `&`, which identifies a product of deconstructors with a destructor for a sum—the De Morgan laws.

Taking `not : Bool -- Bool`, we say

``true not : -- Bool`

---

<sup>1</sup>Inspired by symbols (e.g. ``blue`) in k, which are preceded by symbols (``red`) in Lisp.

``true` has type `-- `true` which can be generalized to `-- `true ⊕ `false` (an acceptable argument to `not : Bool -- Bool`). Sum types generalize but *only on the right*. Compare the situation in linear logic [3][p. 72]

$$\frac{\vdash \Gamma, A \quad \vdash \Gamma, B}{\vdash \Gamma, A \& B} \quad \frac{\vdash \Gamma, A}{\vdash \Gamma, A \oplus B} \quad \frac{\vdash \Gamma, B}{\vdash \Gamma, A \oplus B}$$

Forbidding `&`-generalization on the right manifests as forbidding inexhaustive pattern-matches.<sup>2</sup>

Recalling the identity rule

$$\overline{\vdash A, A^\perp} \quad (\text{Id})$$

and the definition of  $(-)^{\perp}$  by the De Morgan laws  $(A \oplus B)^{\perp} = A^{\perp} \& B^{\perp}$  [2][p. 20], we can prove that `&` (with) generalizes on the left. And indeed a function accepting a value of type  $A \oplus B$  as argument can just as well accept a value of type  $A$ , though this is not manifest in, say, Haskell. But it turns out to be useful; we define `foldr` to work on non-empty lists in subsection 2.2.

## 2.1 Reusing Pattern-Match Arms

We can define a function

```
if : a b `true -- a
    := [ { `true-1 drop } ]
```

This is not so useful on its own—it requires the programmer to supply exactly one value and then discards it—but see how it unifies:

```
else : a b `false -- b
      := [ { `false-1 nip } ]
```

```
choice : a a Bool -- a
        := [ { if & else } ]
```

Pattern-match arms are terms like builtin functions (`+`, `swap`, etc.) and can be reused (named); one enjoys cut elimination for sequences of such.

---

<sup>2</sup>`&` is a product, to form a tuple of type  $(\text{Int}, \text{Int})$  by supplying only one of its constituents would be farcical—yet this is exactly what allowing inexhaustive pattern matches encourages!

## 2.2 Typesafe Head

```
type List a = { `nil @ List(a) a `cons };

type NE a = { List(a) a `cons };

head : NE a -- a
      := [ { `cons-1 nip } ]
```

Any nonempty list can be supplied as an argument to a function on lists. This is far more satisfactory than the situation in Haskell, in which `foldr` &c. must be defined for `NonEmpty` and `List` separately.<sup>3</sup>

## 2.3 Pattern Terms

Suppose we have

```
type Ord = { `lt @ `eq @ `gt };
```

Then we can define

```
lte : { `lt @ `eq } --
      := [ { `lt-1 & `eq-1 } ]

gt : Ord -- Bool
      := [ { lte `false & `gt-1 `true } ]
```

So or-patterns are reusable, enjoy cut-elimination, etc.

## References

- [1] Daniel Ehrenberg. Pattern matching in concatenative programming languages. 2009.
- [2] Jean-Yves Girard. Linear logic. *Theoretical Computer Science*, 50(1):1–101, 1987.
- [3] Jean-Yves Girard. Linear logic: A survey. In Friedrich L. Bauer, Wilfried Brauer, and Helmut Schwichtenberg, editors, *Logic and Algebra of Specification*, pages 63–112, Berlin, Heidelberg, 1993. Springer Berlin Heidelberg.
- [4] Guillaume Munch-Maccagnoni. Focalisation and classical realisability. In Erich Grädel and Reinhard Kahle, editors, *Computer Science Logic*, pages 409–423, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg.

---

<sup>3</sup>Typeclasses allow `foldr` to be called on non-empty lists and lists equally but instances must be defined separately, and the typeclass machinery brings globality of instances and other complications.